Haskell
Home: https://www.haskell.org//
Language spec (2010): https://www.haskell.org/onlinereport/haskell2010/haskell.html
Language spec (1998): https://www.haskell.org/onlinereport//

==========

Basics

Basic syntax

Spec: https://www.haskell.org/onlinereport/haskell2010/haskellch2.html

```
-- A comment
{-
- Multi line comments
-}
---- Identifiers
a = 1         -- must begin with lowercase
_a = 1        -- _ ok
Abc = 1       -- NO.
aBC = 1       -- ok.

---- Literals
-- Numeric
0           -- :: Num a => a
0.1         -- :: Fractional a => a
0o1         -- octal. 1
0O1
0o10        -- 8
0o77        -- 63
0x1         -- hex. 1
0X10        --
0xff        -- 255

-- Character
'a'         -- :: Char. Single character
'\''        -- escape
"ab"        -- :: String.
"\"ab\""    -- escaped
```

Declaration and variables

Spec: https://www.haskell.org/onlinereport/haskell2010/haskellch4.html#x10-620004

```
---- Basic
--
x = 1
X = 1                       -- ERROR. Variable must be lower case

-- type 1) function binding
f x = x + 0
f x y = x + y
g (x::a) = x
```

```
-- type 2) variable
x = 1
x:rest = [1,2,3]            -- pattern   x=1 rest=[1,2,3]
v@x:rest = [1,2,3]          -- v=[1,2,3]

-- type signature
f :: a -> a
f x = x
f :: a -> a; f x = x        -- one line

-- type signature of variable
f (x::Int) = x
f (x::a) = (maxBound::a)


---- where
-- local variables in a declaration
-- i.e. like `let`, but for declaration instead of expression
v = n + m
where
   n = 1
   m = 2
   f :: Num a => a -> a      -- with type signature
   f x = 2 * x
where {n=1; m=2; ... }        -- single line

-- let vs where
v = n where n = 1
v = let n = 1 in n          -- same
```

Pattern guard
Spec: https://www.haskell.org/onlinereport/haskell2010/haskellch4.html#x10-620004

```
-- A simplified case statement
-- Each case is a guard
-- Stop at first match i.e. OR
f x
| x == 0 = "zero"         -- guard x == 0
| x == 1 = "one"          -- guard x == 1
| otherwise = "other"
| True = "other"          -- same
where n = 0               -- where

f x = case () of          -- equivalent `case`
  () | x == 0 -> "zero"
     | x == 1 -> "one"
     ...
  _ -> "error"

-- one liner
f x | x == 0 = "zero" | x == 1 = "one" | ... | otherwise
= "other"       -- one line
f x | x == 0 = "zero" | ... | otherwise = "other" where n=0;m=1
```

```
        -- where

-- multiple guards per case
-- Same as AND
-- i.e. left to right, and stop on first failure
f x, y
 | 1:_ <- x, 1:_ <- y = ...            -- if x[0] = y[0] = 1
 | _:1:_ <- x, _:1:_ <- y = ...        -- if x[1] = y[1] = 1
 | n:_ <- x, n == 1 = ...              -- variable preserved i.e. n
= x[0]
 ...

-- 3 types of guards: pattern, let, boolean
f x
 -- 1) pattern guard
 | 1:_ <- x = ...          -- e.g. list constructor pattern. If x[0]
= 1
 | 2:_ <- x = ...          -- If x[1] = 1
 | [1,2] <- x = ...        -- e.g. list pattern
 | (1,_) <- x = ...        -- e.g. tuple pattern

 -- 2) boolean guard
 | x < 0 = ...
 | x > 1 = ...
 | x > 1, x < 10 = ...          -- same
 | x > 1 && x < 10 = ...
 | True = ...

 -- 3) let declaration.
 -- Always succeeds.
 | let n = 1 = ...              -- always true
 | let n = 1, n > x = ...       -- usage



---- usage: guards in an expression
--
-- Guards are syntax for bindings, not expressions
-- Use an intermediate 'let'
--
-- e.g. in a function call
x = f | ...                      -- ERROR. Not possible
x = let v | ... in f v           -- OK with let

-- e.g. in a do expression
do
  x <- [1..10]
  let y | even x = "even" | otherwise = "odd"
  in return y
```

Expressions
Spec: https://www.haskell.org/onlinereport/haskell2010/haskellch3.html#x8-220003

```
---- Throwing an error
--
error "BAD"        -- like 'throw new Error("BAD")
```

```
*** Exception: BAD
CallStack (from HasCallStack):
  error, called at <interactive>:11:1 in interactive:Ghci1
```

```
undefined         -- like 'throw new Error("undefined")
```

```
CallStack (from HasCallStack):
  error, called at libraries/base/GHC/Err.hs:74:14 in base:GHC.Err
  undefined, called at <interactive>:9:1 in interactive:Ghci1
```

```
---- case expression (if-else)
if x == True then "Good" else "bad"
if x == True then "Good" else 1             -- ERROR. Incompatible
type. error='No instance for (Num String)'
if 1 then "Good" else "bad"                 -- ERROR. Condition not
a Bool.
if x == 1 then "one"                        -- else if
else if x == 2 then "two"
else "other"


---- Case expression
-- i.e. switch-case
case x of
  0 -> "zero"
  1 -> "one"
  _ -> "other"
-- pattern
case x of
(0,_) -> "zero"
(1,_) -> "one"

-- with guards
case x of
 [y]
   | y == 1 -> "one"
   | y == 2 -> "two"
 [_,y]
   | y == 1 -> "one"
   | y == 2 -> "two"

 (_,_,ys)                 -- multiple guards
   | (z:_) <- ys,         -- pattern match
```

```
    z > 3,              -- boolean condition
    let v = z + 1       -- let
      -> ...
 _ -> "other"



---- Parenthesis and Unit
-- i.e.
1
(1)         -- equivalent
()          -- ()



---- let expression
-- expression with local bindings
let x = 2 in x * 3            -- 6
let {x=2;y=3} in x * y        -- one line
let x = 2                     -- multiline
    y = 3 in x * y
let f :: Num a => a -> a              -- with type signature
    f x = a * a
    ...
let f :: Num a => a -> a; f x = a*a   -- one line
    ...

-- lexical scope i.e. variables in order defined
let x = 2
    y = 3 * x        -- y=6
    z = 4 * y        -- z=24
    in x * y * z     -- 288

-- lazy evaluation i.e. let is not evaluated until the expression
is used
a = let x = undefined in 3      -- no error until a is used
```

Pattern matching
Spec: https://www.haskell.org/onlinereport/haskell2010/haskellch3.html#x8-580003.17

Pattern matching
Similar to **destructuring**.
Binding variables based on a pattern.
e.g. bind variable to first value of tuple
e.g. bind to head of a list
e.g. bind to tail of a list

```
---- Pattern matching
--
-- e.g. tuple
(x,y) = (1,2)        -- x=1 y=2
(x,y,z) = (1,2,3)    -- multiple binding. x=1 y=2 z=3
```

```
(x,y,z) = (1,2,3,4)  -- ERROR. arity mismatch
(w,x,y,z) = (1,2,3)  -- ERROR. arity mismatch

-- _: wildcard
-- _ is a special value which can be used multiple times
(x,_,_) = (1,2,3)
(x,a,a) = (1,2,3)    -- ERROR. cannot use same variable except for
_
(_,y,_) = (1,2,3)

-- partial matches
(1,y,z) = (1,2,3)    -- y=2 z=3
(1,2,z) = (1,2,3)    -- z=3
(2,y,z) = (1,2,3)    -- ERROR. Value mismatch
(1,3,z) = (1,2,3)    -- ERROR
n=1
(n,2,z) = (1,2,3)    -- ERROR. Cannot use n as constant. i.e. NOT
(1,2,z)

-- @: whole value binding
-- @ to bind to whole value
v@(x,y,z) = (1,2,3)    -- v=(1,2,3)
v@(1,_,_) = (1,2,3)    -- v=(1,2,3)


-- list pattern matching
[x] = [1]            -- x=1
[x,y,z] = [1,2,3]    -- x=1 y=2 z=3
[1,y] = [1,2]        -- partial match y=2
[x,_,_] = [1,2,3]    -- wildcard
v@[x,y] = [1,2]      -- whole value v=[1,2]


-- Pattern matching with infix constructor
-- e.g. list
x:y:z = [1,2,3]      -- x=1 y=2 z=[3]
x:y:_ = [1,2,3]      -- ignore rest
_:z = [1,2,3]        -- rest only
1:y = [1,2,3]        -- partial match. y=[2,3]
1:y:3:_ = [1,2,3]    -- wildcard y=2 z=[3]
2:y = [1,2,3]        -- ERROR. no match
```

==========

Extensions

Extension: ViewPatterns
GHC: https://ghc.gitlab.haskell.org/ghc/doc/users_guide/exts/view_patterns.html
GHC spec: https://gitlab.haskell.org/ghc/ghc/-/wikis/view-patterns

View patterns
Normal pattern matching matches on just the **given value**.
View patterns matches after applying a **function** on the given value.

GOTCHA: "pattern matches are non-exhaustive" warning
Except for simple cases, compiler can NOT check if pattern is **exhaustive**

Reference: : https://gitlab.haskell.org/ghc/ghc/-/wikis/view-patterns
"Exhaustiveness/redundancy: It is hard to check for completeness of pattern matching; and likewise for overlap. But guards already make both of these hard; and GADTs make completeness tricky too. So matters are not much worse than before."

e.g.

```
{-# LANGUAGE ViewPatterns #-}
{-# OPTIONS_GHC -Wincomplete-patterns #-}          -- enable
incomplete pattern warning

isSumEven :: Int -> Int -> Bool
isSumEven x y = even (x + y)

isEven :: Int -> Bool                             -- OK. No
warning. WHY?
isEven (even -> True) = True
isEven (even -> False) = False

isSumEven2 :: Int -> Int -> Bool                  -- WARNING
"Pattern match(es) are non-exhaustive ..."
isSumEven2 x (isSumEven x -> True) = True
isSumEven2 x (isSumEven x -> False) = False
```

Basic syntax

```
g :: Int -> Bool
f :: Int -> ...
f (g -> True) = ...
f (g -> False) = ...
```

```
-- equivalent
f :: Int -> ...
f x = case g x of
  True -> ...
  False -> ...
```

```
-- multiple variables
g :: Int -> Int -> Bool
f :: Int -> Int -> ...
f x (g x -> True) = ...               -- same as:  f x y = case g
x y of ...
f x (g x -> False) = ...

-- nested
g :: Int -> Bool
f :: Maybe Int -> Bool
f (Just (g -> True)) = ...            -- same as:  f (Just x) =
```

```
case g x of ...
```

Extension: PatternSynonyms
ghc: https://ghc.gitlab.haskell.org/ghc/doc/users_guide/exts/pattern_synonyms.html
gitlab spec: https://gitlab.haskell.org/ghc/ghc/-/wikis/pattern-synonyms

Pattern synonym
Like a data **constructor synonym**
e.g.

```
pattern MyJust a :: a -> Maybe a
pattern MyJust a = Just a                  (MyJust a) is a synonym for Just a
pattern MyJust a <- Just a where           Same explicit
  MyJust a = Just a

MyJust 1 :: Maybe Int                      constructor Just 1
MyJust x = Just 1                          pattern match x = 1
```

Example: pattern with validation
Call validation function as a **view pattern**.

e.g. match even and odd numbers separately

```
{-# LANGUAGE PatternSynonyms #-}
{-# LANGUAGE ViewPatterns #-}

pattern Even :: Integral a => a -> a
pattern Even n <- (\x -> if even x then Just x else Nothing -> Just
n)           -- match only if even

pattern Odd :: Integral a => a -> a
pattern Odd n <- (\x -> if odd x then Just x else Nothing -> Just
n)               -- match only if odd

{-# COMPLETE Even,Odd #-}
                 -- Every number is either Even or Odd

printNum :: (Show a, Integral a) => a -> IO ()
printNum (Even n) = putStrLn $ "Even:" <> show n
printNum (Odd n) = putStrLn $ "Odd:" <> show n

-- e.g.
printNum 2                          -- "Even:2"
printNum 3                          -- "Odd:3"
```

Example: pattern as **smart constructor** i.e. a constructor function for an **abstract data type**
That is, using a constructor function each of the raw data constructor directly.

e.g. a list data type with cached length

```
{-# LANGUAGE PatternSynonyms #-}
{-# LANGUAGE ViewPatterns #-}

data List a = ListEmpty | ListSucc a (List a) Int
  deriving (Show, Eq)

listLength :: List a -> Int
listLength Empty = 0
listLength (ListSucc _ _ n) = n
```

usual smart constructors

```
empty :: List a
empty = ListEmpty

succ :: a -> List a -> List a
succ a ListEmpty = ListSucc a ListEmpty 1
succ a l@(ListSucc _ _ n)  = ListSucc a l (n + 1)
```

pattern smart constructors

```
pattern Empty :: List a
pattern Empty = ListEmpty

pattern Succ :: a -> List a -> List a
pattern Succ a list <- ListSucc a list _
  where
    Succ a l
      | ListSucc _ _ n <- l = ListSucc a l (n + 1)
      | ListEmpty <- l = ListSucc a ListEmpty 1

{-# COMPLETE Empty, Succ #-}
```

usage

```
-- Constructor
Empty :: List Int
Succ 1 $ Succ 2 Empty :: List Int

empty :: List Int                          -- same with usual smart
constructor
succ 1 $ succ 2 empty :: List Int



-- Pattern match
f :: List Int -> ...
f Empty = ...                              -- OK. Can pattern match
f (Succ a l) = ...
f empty = ...                              -- ERROR. Can't pattern
match
```

```
f (succ a l) = ...
```

Example: pattern synonym for type with a **dependent parameter**.
Pattern signature should specify **dependent parameter** only for constructor

e.g.

type File :: FileType -> Type	
File 'Txt	text file
File 'Log	log file
pattern TxtFile :: FilePath -> File 'Txt	BAD. Can only pattern match File 'Txt
pattern TxtFile :: () => (t ~ 'Txt) => FilePath -> File t	GOOD any File t when pattern matching File 'Txt when constructing

```
{-# LANGUAGE DataKinds #-}
{-# LANGUAGE GADTs #-}
{-# LANGUAGE KindSignatures #-}
{-# LANGUAGE StandaloneKindSignatures #-}
{-# LANGUAGE PatternSynonyms #-}

import Data.Kind (Type)

data FileType = Txt | Log

type SFileType :: FileType -> Type
data SFileType filetype where
  STxt :: SFileType 'Txt
  SLog :: SFileType 'Log

type File :: FileType -> Type
data File filetype = File (SFileType filetype) FilePath

-- pattern TxtFile :: FilePath -> File 'Txt       -- BAD. Cannot
pattern match any File
pattern TxtFile ::                                -- GOOD
  forall (filetype :: FileType).
  () =>                                           -- match any File
  (filetype ~ 'Txt) =>                            -- construct File
'Txt
  FilePath ->
  File filetype
pattern TxtFile p = File STxt p

-- pattern LogFile :: FilePath -> File 'Log       -- BAD. Cannot
pattern match any File
pattern LogFile ::                                -- GOOD
  forall (filetype :: FileType).
  () =>                                           -- match any File
  (filetype ~ 'Log) =>                            -- construct File
'Log
```

```
  FilePath ->
  File filetype
pattern LogFile p = File SLog p

{-# COMPLETE TxtFile, LogFile #-}
```

```
-- e.g. construct with synonym
textFile :: File 'Txt
textFile = TxtFile "a.txt"

logFile :: File 'Log
logFile = LogFile "a.log"

-- e.g. match with synonym
instance Show (File filetype) where
  show (TxtFile path) = "TxtFile " <> path
  show (LogFile path) = "LogFile " <> path
```

Syntax

```
---- Basics
--
-- Basic
pattern A a = Just a          -- synonym for Just a
pattern A a b = (a, b)        -- multiple variable
pattern A (a, a) = a          -- ERROR. Variables only on left side
```

```
-- e.g.
pattern A a = Just a
A a = Just 1              -- a = 1
A a = Nothing             -- ERROR. Doesn't match pattern

x = A 1 :: Maybe Int
x = Just 1                -- same
```

```
-- Pattern only (unidirectional)
pattern A a <- Just a         --
pattern A a <- (f -> a)       -- match with view pattern  f :: b ->
a
```

```
-- e.g.
pattern A :: Int -> Int
pattern A a <- ((*2) -> a)

A a = 1                       -- a = 2
```

```
-- Explicit constructor (explicitly bidirectional)
pattern A :: Int -> Maybe Int
pattern A a <- Just a where           -- explicit
  A a = Just a
pattern A a = Just a                  -- same

pattern A a <- Just a where           --
  A a
    | a > 0 = Just a
    | otherwise = Nothing


-- Pattern signature
--
pattern A :: a -> Maybe a
pattern A a = Just a
pattern A :: () => () => a -> Maybe a          -- same
pattern A ::
  Num a =>                                     -- Num a => Maybe a
  () =>                                        -- additional
constraints. In particular, constraints when matching a GADT
constructor.
  a -> Maybe a
pattern A :: () => Num a => a -> Maybe a       -- ERROR
```

```
-- e.g. GADT constraints
data MyMaybe a where
  MyJust :: Show a => a -> MyMaybe a
  MyNothing :: MyMaybe a

pattern A :: () => Show a => a -> MyMaybe a         -- GOOD
pattern A :: () => () => a -> MyMaybe a             -- ERROR. Need
(Show a) for MyJust
pattern A a = MyJust a

pattern B :: Show a => () => a -> MyMaybe a         -- BAD.
Compiles but doesn't work as expected.
                                                    -- Show a =>
MyMaybe a

data X
A x = MyNothing :: MyMaybe Int                 -- OK
A x = MyNothing :: MyMaybe X                   -- OK

B x = MyNothing :: MyMaybe Int                 -- OK
B x = MyNothing :: MyMaybe X                   -- ERROR since X is
not Show "No instance for (Show (Int -> Int)) arising from a use of
‘B’"
```

==========

OLD

```
-- e.g.
pattern A a b <- [a, b] where
  A a b = [b, a]

-- pattern
A a b = [1, 2]                    -- OK. a=1 b=2
A a b = [1]                       -- ERROR. Doesn't match pattern
A a b = [1, 2, 3]                 -- ERROR. Doesn't match pattern

-- constructor
A 1 2                             -- [2, 1]
```

```
-- Bidirectional shorthand
pattern A a b = [a, b]
pattern A a b <- [a, b] where          -- same
  A a b = [a, b]
pattern A a = (id -> a)                -- ERROR. No view pattern
```